

Question 1: Product of Array except self

Given an integer array `nums`, return an array `answer` such that `answer[i]` is equal to the product of all the elements of `nums` except `nums[i]`.

The product of any prefix or suffix of `nums` is guaranteed to fit in a 32-bit integer.

You must write an algorithm that runs in $O(n)$ time and without using the division operation.

Example 1:

Input: `nums = [1,2,3,4]`

Output: `[24,12,8,6]`

Example 2:

Input: `nums = [-1,1,0,-3,3]`

Output: `[0,0,9,0,0]`

Constraints:

$2 \leq \text{nums.length} \leq 105$

$-30 \leq \text{nums}[i] \leq 30$

The input is generated such that `answer[i]` is guaranteed to fit in a 32-bit integer.

Approach :Optimal

- Create **prefix** and **suffix** arrays
- Build prefix products from left to right
- Build suffix products from right to left
- Multiply prefix and suffix for each index



```
public int[] productExceptSelf(int[] nums) {  
    int n = nums.length;  
    int[] prefix = new int[n];  
    int[] suffix = new int[n];  
    int[] answer = new int[n];  
    prefix[0] = 1;  
    for (int i = 1; i < n; i++) {  
        prefix[i] = prefix[i - 1] * nums[i - 1];  
    }  
    suffix[n - 1] = 1;  
    for (int i = n - 2; i >= 0; i--) {  
        suffix[i] = suffix[i + 1] * nums[i + 1];  
    }  
    for (int i = 0; i < n; i++) {  
        answer[i] = prefix[i] * suffix[i];  
    }  
    return answer;  
}
```

- Time: $O(n)$
- Space: $O(n)$



Question 2: Longest Consecutive Sequence

Given an array of integers `nums`, return the length of the longest consecutive sequence of elements that can be formed.

A consecutive sequence is a sequence of elements in which each element is exactly 1 greater than the previous element. The elements do not have to be consecutive in the original array.

You must write an algorithm that runs in $O(n)$ time.

Example 1:

Input: `nums = [2,20,4,10,3,4,5]`

Output: 4

Explanation: The longest consecutive sequence is [2, 3, 4, 5].

Example 2:

Input: `nums = [0,3,2,5,4,6,1,1]`

Output: 7

Constraints:

$0 \leq \text{nums.length} \leq 1000$

$-10^9 \leq \text{nums}[i] \leq 10^9$

Approach :Brute force

For every number, try to build a consecutive sequence by checking if the next numbers exist.

```
2 public int longestConsecutive(int[] nums) {
3     int maxLen = 0;
4     for (int i = 0; i < nums.length; i++) {
5         int current = nums[i];
6         int count = 1;
7         boolean found = true;
8         while (found) {
9             found = false;
10            for (int j = 0; j < nums.length; j++) {
11                if (nums[j] == current + 1) {
12                    current++;
13                    count++;
14                    found = true;
15                    break;
16                }
17            }
18        }
19        maxLen = Math.max(maxLen, count);
20    }
21    return maxLen;
22 }
```

- **Time:** $O(n^2)$ (searching existence linearly)
- **Space:** $O(1)$

Approach :Better

- Sort the array first.
- Then count consecutive numbers while skipping duplicates.

```
2 public int longestConsecutive(int[] nums) {
3     if (nums.length == 0) return 0;
4
5     Arrays.sort(nums);
6     int maxLen = 1;
7     int currentLen = 1;
8
9     for (int i = 1; i < nums.length; i++) {
10        if (nums[i] == nums[i - 1]) {
11            continue; // skip duplicates
12        } else if (nums[i] == nums[i - 1] + 1) {
13            currentLen++;
14        } else {
15            currentLen = 1;
16        }
17        maxLen = Math.max(maxLen, currentLen);
18    }
19    return maxLen;
20 }
```

- **Time:** $O(n \log n)$
- **Space:** $O(1)$ (ignoring sorting internals)

Approach :Optimal

- Store all numbers in a **HashSet**
- Only start counting when the number is the **start of a sequence**

```
2  ✓ public int longestConsecutive(int[] nums) {  
3      Set<Integer> set = new HashSet<>();  
4  ✓      for (int num : nums) {  
5          set.add(num);  
6      }  
7      int maxLen = 0;  
8  ✓      for (int num : set) {  
9  ✓          if (!set.contains(num - 1)) {  
10         int current = num;  
11         int count = 1;  
12  ✓         while (set.contains(current + 1)) {  
13             current++;  
14             count++;  
15         }  
16         maxLen = Math.max(maxLen, count);  
17     }  
18 }  
19     return maxLen;  
20 }
```

- Time: $O(n)$
- Space: $O(n)$

Question 3: Valid Palindrome

Given a string `s`, return `true` if it is a palindrome, otherwise return `false`. A palindrome is a string that reads the same forward and backward. It is also case-insensitive and ignores all non-alphanumeric characters. Note: Alphanumeric characters consist of letters (A-Z, a-z) and numbers (0-9).

Example 1:

Input: `s = "Was it a car or a cat I saw?"`

Output: `true`

Explanation: After considering only alphanumerical characters we have "wasitacaroracatisaw", which is a palindrome.

Example 2:

Input: `s = "tab a cat"`

Output: `false`

Explanation: "tabacat" is not a palindrome.

Constraints:

$1 \leq s.length \leq 1000$

`s` is made up of only printable ASCII characters.

Approach 1: Brute force

- Clean the string by:

Removing non-alphanumeric characters

Converting to lowercase

- Reverse the cleaned string
- Compare original cleaned string with its reverse

```
2 ✓ public boolean isPalindrome(String s) {  
3     StringBuilder clean = new StringBuilder();  
4  
5 ✓     for (char c : s.toCharArray()) {  
6 ✓         if (Character.isLetterOrDigit(c)) {  
7             clean.append(Character.toLowerCase(c));  
8         }  
9     }  
10  
11     String original = clean.toString();  
12     String reversed = clean.reverse().toString();  
13  
14     return original.equals(reversed);  
15 }
```

- Time: $O(n)$
- Space: $O(n)$

Approach 2: Optimal

- Use **two pointers** directly on the original string
- Skip non-alphanumeric characters
- Compare lowercase characters from both ends



```
2   boolean nonAlphaNum(char c) {
3       return !(Character.isLetterOrDigit(c));
4   }
5   public boolean isPalindrome(String s) {
6       int l=0, r=s.length()-1;
7       while(l < r) {
8           if(nonAlphaNum(s.charAt(l)))
9               l++;
10          else if(nonAlphaNum(s.charAt(r)))
11              r--;
12          else {
13              if(Character.toLowerCase(s.charAt(l)) != Character.toLowerCase(s.charAt(r)))
14                  return false;
15              l++;
16              r--;
17          }
18      }
19      return true;
20  }
```

- Time: $O(n)$
- Space: $O(1)$





Question 4: Two Integer Sum II

Given an array of integers numbers that is sorted in non-decreasing order.

Return the indices (1-indexed) of two numbers, [index1, index2], such that they add up to a given target number target and $\text{index1} < \text{index2}$. Note that index1 and index2 cannot be equal, therefore you may not use the same element twice.

There will always be exactly one valid solution.

Your solution must use
 $O(1)$, $O(1)$ additional space.

Example 1:

Input: numbers = [1,2,3,4], target = 3

Output: [1,2]

Explanation:

The sum of 1 and 2 is 3. Since we are assuming a 1-indexed array, $\text{index1} = 1$, $\text{index2} = 2$. We return [1, 2].

Constraints:

$2 \leq \text{numbers.length} \leq 1000$

$-1000 \leq \text{numbers}[i] \leq 1000$

$-1000 \leq \text{target} \leq 1000$

Approach :

Use two pointers:

- **left** at the beginning
- **right** at the end

Compare **numbers[left] + numbers[right]** with **target**

- If sum is too small → move **left**
- If sum is too large → move **right**

```
2    public int[] twoSum(int[] numbers, int target) {
3        int l = 0;
4        int r = numbers.length-1;
5        int[] ans = new int[2];
6        while(l < r) {
7            int sum = numbers[l]+numbers[r];
8            if(sum == target) {
9                ans[0] = l+1;
10               ans[1] = r+1;
11               break;
12           }
13           else if(sum > target)
14               r--;
15           else
16               l++;
17       }
18       return ans;
19   }
20 }
```

- **Time:** $O(n)$
- **Space:** $O(1)$



Question 5: 3 Sum

Given an integer array `nums`, return all the triplets `[nums[i], nums[j], nums[k]]` where `nums[i] + nums[j] + nums[k] == 0`, and the indices `i`, `j` and `k` are all distinct.

The output should not contain any duplicate triplets. You may return the output and the triplets in any order.

Example 1:

Input: `nums = [-1,0,1,2,-1,-4]`

Output: `[[-1,-1,2],[-1,0,1]]`

Explanation:

$\text{nums}[0] + \text{nums}[1] + \text{nums}[2] = (-1) + 0 + 1 = 0.$

$\text{nums}[1] + \text{nums}[2] + \text{nums}[4] = 0 + 1 + (-1) = 0.$

$\text{nums}[0] + \text{nums}[3] + \text{nums}[4] = (-1) + 2 + (-1) = 0.$

The distinct triplets are `[-1,0,1]` and `[-1,-1,2]`.

Constraints:

$3 \leq \text{nums.length} \leq 1000$

$-10^5 \leq \text{nums}[i] \leq 10^5$

Approach :Brute force

- Try **all possible triplets** and check if their sum is 0.
- Use a set to avoid duplicate triplets.

```
2 ✓ public List<List<Integer>> threeSum(int[] nums) {  
3     Set<List<Integer>> hs = new HashSet();  
4     int n = nums.length;  
5 ✓     for(int i=0; i<n; i++) {  
6 ✓         for(int j=i+1; j<n; j++) {  
7 ✓             for(int k=j+1; k<n; k++) {  
8 ✓                 if(nums[i]+nums[j]+nums[k] == 0) {  
9                     List<Integer> lst = new ArrayList();  
10                    lst.add(nums[i]);  
11                    lst.add(nums[j]);  
12                    lst.add(nums[k]);  
13                    Collections.sort(lst);  
14                    hs.add(lst);  
15                }  
16            }  
17        }  
18    }  
19    return new ArrayList(hs);  
20 }
```

- **Time:** $O(n^3)$
- **Space:** $O(n)$ (for storing results)

Approach :Optimal

- Sort the array
- Fix one number **i**
- Use **two pointers** (**left**, **right**) to find pairs that sum to $-nums[i]$
- Skip duplicates to avoid repeated triplets

```
2 public List<List<Integer>> threeSum(int[] nums) {
3     List<List<Integer>> result = new ArrayList<>();
4     Arrays.sort(nums);
5     int n = nums.length;
6
7     for (int i = 0; i < n - 2; i++) {
8         if (i > 0 && nums[i] == nums[i - 1]) continue;
9         int left = i + 1;
10        int right = n - 1;
11        while (left < right) {
12            int sum = nums[i] + nums[left] + nums[right];
13            if (sum == 0) {
14                result.add(Arrays.asList(nums[i], nums[left], nums[right]));
15                while (left < right && nums[left] == nums[left + 1]) left++;
16                while (left < right && nums[right] == nums[right - 1]) right--;
17                left++;
18                right--;
19            } else if (sum < 0) {
20                left++;
21            } else {
22                right--;
23            }
24        }
25    }
26    return result;
27 }
```

- Time: $O(n^2)$
- Space: $O(1)$ (excluding output)